

Guia de Instalação e Configuração via Docker: O-CU e O-DU (OCUDU)

Grupo de Pesquisa em Telecomunicações em Fio (GTEL)

8 de julho de 2026

Conteúdo

1	Introdução	2
2	Pré-requisitos	2
3	Cenário 1: Instalação Centralizada (All-in-One)	2
3.1	Preparação do Ambiente e Dependências	2
3.2	Estrutura do Docker Compose	2
3.3	Inicialização do Ambiente	5
4	Cenário 2: Instalação Descentralizada	5
4.1	Topologia de Rede	5
4.2	Configuração do Host A (5G Core)	5
4.3	Configuração do Host B (gNB - OCUDU)	6
5	Monitoramento e Validação	6
6	Configurações para o OCUDU GNB+CORE	7
6.1	Repositório ocudu_core	7
6.2	Configurando arquivo yml para open5gs	7
6.3	Alterando o arquivo open5gs.env	10
6.4	Alterando o arquivo open5gs_entrypoint.sh	10
6.5	Iniciando container open5gs	12
6.6	Repositório ocudu_gnb	12
6.7	Configurando arquivo yml para gnb	12
6.8	Alterando arquivo .env (GRAFANA)	13

1 Introdução

Este documento descreve o processo passo a passo de implantação dos componentes O-CU (Central Unit) e O-DU (Distributed Unit) utilizando virtualização baseada em containers. A abordagem via Docker garante isolamento, reprodutibilidade e agilidade no deploy da rede 5G Standalone. O guia aborda dois cenários: uma instalação centralizada (All-in-One) e uma descentralizada (Core e gNB distribuídos em hosts distintos).

2 Pré-requisitos

Antes de iniciar o deploy, certifique-se de que os hosts possuem o ambiente Docker devidamente configurado:

- Docker Engine (versão 20.10+ recomendada).
- Docker Compose V2 instalado.
- Permissão para executar o Docker sem *sudo* (opcional, mas recomendado para o usuário local).
- Acesso à internet para realizar o *pull* das imagens ou conectividade de rede configurada entre as máquinas.

3 Cenário 1: Instalação Centralizada (All-in-One)

Neste cenário, todas as funções do 5G Core e da gNB (OCUDU) são instanciadas na mesma máquina hospedeira, de modo que os containers comunicam-se internamente através de uma rede Docker dedicada em bridge.

3.1 Preparação do Ambiente e Dependências

O primeiro passo é clonar o repositório oficial do projeto e suas dependências, conforme detalhado no site oficial do OCUDU (https://docs.ocudu.org/user_manual/installation/). Em relação à biblioteca de processamento FFT, utilizaremos a *libfftw*, que é essencial para calcular as transformadas rápidas de Fourier usadas na modulação e demodulação OFDM. Além disso, como a gNB estará em execução nesta máquina e precisará acessar o hardware, é necessário instalar os drivers do rádio. Utilizaremos o UHD (USRP Hardware Driver), disponível no repositório da Ettus Research (<https://github.com/EttusResearch/uhd>).

3.2 Estrutura do Docker Compose

Após a preparação dos drivers e repositórios locais, navegue até o diretório `ocudu/docker/`. É neste local que se encontra o arquivo `docker-compose.yml` oficial, responsável por gerenciar o Core (Open5GS) e a gNodeB simultaneamente.

```
1 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
   #   Systems Limited
2 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
3
```

```

4 services:
5   5gc:
6     container_name: open5gs_5gc
7     build:
8       context: open5gs
9       target: open5gs
10      args:
11        OS_VERSION: "22.04"
12        OPEN5GS_VERSION: "v2.7.6"
13      env_file:
14        - ${OPEN_5GS_ENV_FILE:-open5gs/open5gs.env}
15      privileged: true
16      ports:
17        - "9999:9999/tcp"
18      # Uncomment port to use the 5gc from outside the docker
19      #   network
20      # - "38412:38412/sctp"
21      # - "2152:2152/udp"
22      command: 5gc -c open5gs-5gc.yml
23      volumes:
24        - ./open5gs/subscriber_db.csv:/open5gs/subscriber_db.csv:ro
25      healthcheck:
26        test: ["CMD-SHELL", "nc -z 127.0.0.20 7777"]
27        interval: 3s
28        timeout: 1s
29        retries: 60
30      networks:
31        ran:
32          ipv4_address: ${OPEN5GS_IP:-10.53.1.2}
33
34   gnb:
35     container_name: ocudu_gnb
36     # Build info
37     image: ocudu/gnb
38     build:
39       context: ..
40       dockerfile: docker/Dockerfile
41       args:
42         OS: ${OS:-ubuntu}
43         OS_VERSION: ${OS_VERSION:-26.04}
44     # privileged mode is required only for accessing usb devices
45     privileged: true
46     # Extra capabilities always required
47     cap_add:
48       - SYS_NICE
49       - CAP_SYS_PTRACE
50     volumes:
51       # Access USB to use some SDRs
52       - /dev/bus/usb/:/dev/bus/usb/
53       # Sharing images between the host and the pod.
54       - /usr/share/uhd/images:/usr/share/uhd/images

```

```

54     # Save logs and more into gnb-storage
55     - gnb-storage:/tmp
56   configs:
57     - gnb_config.yml
58     - gnb_compose_config.yml
59   networks:
60     ran:
61       ipv4_address: ${GNB_IP:-10.53.1.3}
62     metrics:
63       ipv4_address: 172.19.1.3
64   # Start GNB container after 5gc is up and running
65   depends_on:
66     5gc:
67       condition: service_healthy
68   # Command to run into the final container
69   command: gnb -c /gnb_config.yml -c /gnb_compose_config.yml
70
71 configs:
72   gnb_config.yml:
73     file: ${GNB_CONFIG_PATH:-../configs/gnb_rf_b200_tdd_n78_20mhz
74       .yml}
75   gnb_compose_config.yml:
76     content: |
77       cu_cp:
78         amf:
79           addrs: ${OPEN5GS_IP:-10.53.1.2}
80           bind_addrs: ${GNB_IP:-10.53.1.3}
81         metrics:
82           autostart_stdout_metrics: true
83           enable_json: true
84         remote_control:
85           bind_addr: 0.0.0.0
86           enabled: true
87
88 volumes:
89   gnb-storage:
90
91 networks:
92   ran:
93     ipam:
94       driver: default
95       config:
96         - subnet: 10.53.1.0/24
97   metrics:
98     ipam:
99       driver: default
100      config:
101        - subnet: 172.19.1.0/24

```

Listing 1: docker-compose.yml oficial centralizado

3.3 Inicialização do Ambiente

Para subir a stack completa do Cenário 1, execute o comando abaixo no diretório `ocudu/docker/`:

```
1 # Inicializar os containers
2 docker compose up
3
4 # Verificar o status dos containers ativos
5 docker ps
```

Listing 2: Comandos para execução no Host Único

4 Cenário 2: Instalação Descentralizada

Na arquitetura distribuída, o ambiente é dividido para simular um cenário real de Open-RAN. O 5G Core rodará no **Host A** e a gNB (OCUDU) rodará no **Host B**. A comunicação entre os containers agora trafega pela rede física subjacente.

4.1 Topologia de Rede

- **Host A (5G Core):** IP Físico 192.168.10.100
- **Host B (gNB - OCUDU):** IP Físico 192.168.10.200

4.2 Configuração do Host A (5G Core)

No Host A, o container do Core precisa expor as portas de controle e plano de usuário para a interface de rede física, permitindo conexões externas da gNB.

```
1 version: '3.8'
2
3 services:
4   5gc:
5     container_name: open5gs_5gc
6     build:
7       context: open5gs
8       target: open5gs
9     privileged: true
10    ports:
11      - "9999:9999/tcp"
12      - "38412:38412/sctp" # Descomentado: Porta padrão do AMF (
13                          #   NGAP)
14      - "2152:2152/udp"    # Descomentado: Porta do GTP-U (Plano
15                          #   de Usuário)
16    command: 5gc -c open5gs-5gc.yml
17    volumes:
18      - ./open5gs/subscriber_db.csv:/open5gs/subscriber_db.csv:ro
19    restart: always
```

Listing 3: docker-compose.yml do Core (Host A)

Para iniciar o serviço no Host A:

```
1 docker compose up -d
```

Listing 4: Execução do Core no Host A

4.3 Configuração do Host B (gNB - OCUDU)

No Host B, utilizaremos o modo de rede `host` para que a gNB se comunique diretamente na rede física e alcance o Core (Host A), mantendo os privilégios de hardware exigidos pelo UHD.

```
1 version: '3.8'
2
3 services:
4   gnb:
5     container_name: ocudu_gnb
6     image: ocudu/gnb
7     privileged: true
8     cap_add:
9       - SYS_NICE
10      - CAP_SYS_PTRACE
11     volumes:
12       - /dev/bus/usb/:/dev/bus/usb/
13       - /usr/share/uhd/images:/usr/share/uhd/images
14       - gnb-storage:/tmp
15     network_mode: "host"           # Permite acesso direto as
16                                     interfaces fisicas
17     restart: always
18     # As configuracoes devem apontar o AMF para 192.168.10.100
19
20 volumes:
21   gnb-storage:
```

Listing 5: docker-compose.yml da gNB (Host B)

Para iniciar a gNB no Host B:

```
1 docker compose up -d
```

Listing 6: Execução da gNB no Host B

5 Monitoramento e Validação

Para validar se a associação NGAP entre o container do OCUDU e o container do Core foi estabelecida com sucesso, utilize as ferramentas de inspeção de logs e redes do Docker.

```
1 # Verificar os logs em tempo real da gNB no Host B
2 docker logs -f ocudu_gnb
3
4 # Inspeccionar estatísticas de rede do container do Core no Host
5   A
6 docker stats open5gs_5gc
```

6 Configurações para o OCUDU GNB+CORE

Após a instalação do projeto OCUDU é necessário realizar algumas configurações para a implantação da rede 5G de forma separada.

Primeiramente, é necessário escolher o host que será trabalhado. Dessa forma, o tutorial terá início com as modificações nos arquivos do host que irá hospedar o container do open5gc.

6.1 Repositório ocudu_core

Renomeie o repositório para “ocudu_core”:

```
1 mv ocudu ocudu_core
```

Ademais, é necessário acessar a pasta do repositório instalado:

```
1 cd ocudu_core/docker
```

Liste os arquivos do repositório com `ls` e verifique os arquivos do repositório. Dentro da pasta `docker`, existe o arquivo “docker-compose.yml”:

```
1 sdr@epc-server:~/ocudu_core/docker$ ls
2 Dockerfile docker-compose.split.yml docker-compose.yml open5gs
  telegraf
3 README.md docker-compose.ui.yml grafana scripts
```

6.2 Configurando arquivo yml para open5gs

Abrindo o arquivo em questão:

```
1 nano docker-compose.yml
```

Teremos então a saída:

```
1 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
  Systems Limited
2 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
3 services:
4   5gc:
5     container_name: open5gs_5gc
6     build:
7       context: open5gs
8       target: open5gs
9     args:
10       OS_VERSION: "22.04"
11       OPEN5GS_VERSION: "v2.7.6"
12     env_file:
13       - ${OPEN_5GS_ENV_FILE:-open5gs/open5gs.env}
14     privileged: true
```

```

15     ports:
16         - "9999:9999/tcp"
17     # Uncomment port to use the 5gc from outside the docker
18         network
19     # - "38412:38412/sctp"
20     # - "2152:2152/udp"
21     command: 5gc -c open5gs-5gc.yml
22     volumes:
23         - ./open5gs/subscriber_db.csv:/open5gs/subscriber_db.csv:ro
24     healthcheck:
25         test: ["CMD-SHELL", "nc -z 127.0.0.20 7777"]
26         interval: 3s
27         timeout: 1s
28         retries: 60
29     networks:
30         ran:
31             ipv4_address: ${OPEN5GS_IP:-10.53.1.2}
32     ..[codigo continua]..

```

Se atendo ao serviço 5gc, para que o container fique exposto à rede do host, é necessário acrescentar o modo de rede igual ao do host:

```

1 network_mode: host

```

Alterando:

```

1 services:
2     5gc:
3         container_name: open5gs_5gc
4         network_mode: host
5         build:
6         ..[codigo continua]..

```

Incluindo variável HOSTNAME:

```

1     privileged: true
2     environment:
3         - HOSTNAME=0.0.0.0
4     ports:..[codigo continua]..
5     gnb:
6         container_name: ocudu_gnb
7         # Build info
8         image: ocudu/gnb
9         build:
10         ..[codigo continua]..

```

Por último, a seção networks deve ser comentada:

```

1     healthcheck:
2         test: ["CMD-SHELL", "nc -z 127.0.0.20 7777"]
3         interval: 3s
4         timeout: 1s
5         retries: 60
6     # networks:
7     #     ran:

```



```
8      #      ipv4_address: ${OPEN5GS_IP:-10.53.1.2}
9      ..[codigo continua]..
```

No fim do arquivo existe a declaração de criação de redes `network` e `metrics`, que devem ser comentadas visto o container estar em modo `host`:

```
1  #networks:
2  #    ran:
3  #      ipam:
4  #        driver: default
5  #        config:
6  #          - subnet: 10.53.1.0/24
7  #    metrics:
8  #      ipam:
9  #        driver: default
10 ##      config:
11 #        - subnet: 172.19.1.0/24
12 ..[codigo continua]..
```

As modificações ficam dessa forma:

```
1  services:
2    5gc:
3      container_name: open5gs_5gc
4      network_mode: host
5      build:
6          context: open5gs
7          target: open5gs
8          args:
9              OS_VERSION: "22.04"
10             OPEN5GS_VERSION: "v2.7.6"
11      env_file:
12          - ${OPEN_5GS_ENV_FILE:-open5gs/open5gs.env}
13      privileged: true
14      environment:
15          - HOSTNAME=0.0.0.0
16      ports:
17          - "9999:9999/tcp"
18      # Uncomment port to use the 5gc from outside the docker
19      # network
20      # - "38412:38412/sctp"
21      # - "2152:2152/udp"
22      command: 5gc -c open5gs-5gc.yml
23      volumes:
24          - ./open5gs/subscriber_db.csv:/open5gs/subscriber_db.csv:ro
25      healthcheck:
26          test: ["CMD-SHELL", "nc -z 127.0.0.20 7777"]
27          interval: 3s
28          timeout: 1s
29          retries: 60
30      # networks:
31      #    ran:
32      #      ipv4_address: ${OPEN5GS_IP:-10.53.1.2}
```

32 ..[codigo continua]..

6.3 Alterando o arquivo open5gs.env

Além de modificar o arquivo `docker-compose.yml`, é necessário mudar outro arquivo chamado `open5gs.env`.

Para acessá-lo:

```
1 cd ocudu_core/docker/open5gs
2 nano open5gs.env
```

A saída do comando será:

```
1 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
  Systems Limited
2 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
3 MONGODB_IP=127.0.0.1
4 OPEN5GS_IP=10.53.1.2
5 UE_IP_BASE=10.45.0
6 UPF_ADVERTISE_IP=10.53.1.2
7 DEBUG=false
8 SUBSCRIBER_DB=001010123456780,00112233445566778899aabbccddeeff,
  opc,63bfa50ee6523365ff14c1f45f88737d,8000,9,10.45.1.2
9 NETWORK_NAME_FULL=OCUDU
10 NETWORK_NAME_SHORT=OCUDU
11 # Timezone - this setting will also be conveyed by Open5GS
  towards UE via "configuration update command"; comment for UTC
12 TZ=Europe/Madrid
```

Será necessário mudar os espaços `OPEN5GS_IP` e `UPF_ADVERTISE_IP`:

```
1 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
  Systems Limited
2 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
3 MONGODB_IP=127.0.0.1
4 OPEN5GS_IP=192.168.152.131
5 UE_IP_BASE=10.45.0
6 UPF_ADVERTISE_IP=192.168.152.131
7 DEBUG=false
8 SUBSCRIBER_DB=001010000000008,465B5CE8B199B49FAA5F0A2EE238A6BD,
  opc,E8ED289DEBA952E4283B54E88E6183CB,8000,9,10.45.1.2
9 NETWORK_NAME_FULL=OCUDU
10 NETWORK_NAME_SHORT=OCUDU
11 # Timezone - this setting will also be conveyed by Open5GS
  towards UE via "configuration update command"; comment for UTC
12 TZ=America/Fortaleza
```

No caso, como o modo de rede está configurado para host, `OPEN5GS_IP` é o mesmo IP do host.

6.4 Alterando o arquivo open5gs_entrpoint.sh

Para acessar o arquivo:

```
1 cd ocudu_core/docker/open5gs
2 nano open5gs_entrypoint.sh
```

Na saída:

```
1 #! /bin/bash
2 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
  Systems Limited
3 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
4 export UE_GATEWAY_IP="${UE_IP_BASE}.1"
5 export UE_IP_RANGE="${UE_IP_BASE}.0/24"
6 INSTALL_ARCH=x86_64-linux-gnu
7 if [ "$(uname -m)" = "aarch64" ]; then
8     INSTALL_ARCH="aarch64-linux-gnu"
9 fi
10 export INSTALL_ARCH
11 envsubst < open5gs-5gc.yml.in > open5gs-5gc.yml
12 # create dummy interfaces on localhost ip range for open5gs
  entities to bind to
13 for IP in {2..22}
14 do
15     ip link add name lo$IP type dummy
16     ip ad ad 127.0.0.$IP/24 dev lo$IP
17     ip link set lo$IP up
18 done
19 ..[codigo continua]..
```

No bloco onde as interfaces de rede dummy são criadas, é necessário mudar a máscara CIDR de 24 para 32. Ademais, deve ser feita a adição de rotas e habilitar o encaminhamento, dessa forma os UEs irão conseguir ter acesso à internet.

```
1 #! /bin/bash
2 # SPDX-FileCopyrightText: Copyright (C) 2021-2026 Software Radio
  Systems Limited
3 # SPDX-License-Identifier: BSD-3-Clause-Open-MPI
4 export UE_GATEWAY_IP="${UE_IP_BASE}.1"
5 export UE_IP_RANGE="${UE_IP_BASE}.0/24"
6 INSTALL_ARCH=x86_64-linux-gnu
7 if [ "$(uname -m)" = "aarch64" ]; then
8     INSTALL_ARCH="aarch64-linux-gnu"
9 fi
10 export INSTALL_ARCH
11 sysctl -w net.ipv4.ip_forward=1
12 sysctl -w net.ipv6.conf.all.forwarding=1
13 iptables -t nat -A POSTROUTING -s 10.45.0.0/16 ! -o ogstun -j
  MASQUERADE
14 ip6tables -t nat -A POSTROUTING -s 2001:db8:cafe::/48 ! -o ogstun
  -j MASQUERADE
15 iptables -A FORWARD -i ogstun -j ACCEPT
16 iptables -A FORWARD -o ogstun -m state --state RELATED,
  ESTABLISHED -j ACCEPT
17 envsubst < open5gs-5gc.yml.in > open5gs-5gc.yml
```

```

18 # create dummy interfaces on localhost ip range for open5gs
    entities to bind to
19 for IP in {2..22}
20 do
21     ip link add name lo$IP type dummy
22     ip ad #ad 127.0.0.$IP/32 dev lo$IP
23     ip link set lo$IP up
24 done
25 ..[codigo continua]..

```

6.5 Iniciando container open5gs

Para inicializar o container do open5gs, basta estar na pasta docker e aplicar o comando:

```

1 cd ocudu_core/docker
2 sudo docker compose up 5gc

```

Para verificar se o container está online basta utilizar `sudo docker ps`.

6.6 Repositório ocudu_gnb

Nessa etapa do tutorial, o foco é mudado para o host que hospedará o container da gnb. Dessa forma, primeiramente o nome do repositório ocudu baixado deve ser mudado para `ocudu_gnb`.

```

1 mv ocudu ocudu_gnb

```

Agora, para a próxima etapa, acesse a pasta docker:

```

1 cd ocudu_gnb/docker

```

6.7 Configurando arquivo yml para gnb

Acessando o arquivo `docker-compose.yml`:

```

1 nano docker-compose.yml

```

É necessário que o container da gnb, assim como o container do open5gs, funcionem em modo de rede host. Dessa forma, é preciso acrescentar ao arquivo a configuração `network_mode: host`.

```

1 ..[codigo continua]..
2 gnb:
3     container_name: ocudu_gnb
4     network_mode: host
5     # Build info
6     image: ocudu/gnb
7     build:
8     ..[codigo continua]..

```

Além disso, os endereços IPs (`amf: addrs`, `bind_addrs`) presentes no arquivo devem ser mudados de acordo com o endereço do host de cada container. Assim, como feito no arquivo yml para o 5gc, as declarações de rede devem ser comentadas.

```

1 networks:
2   ran:
3     ipv4_address: ${GNB_IP:-10.53.1.3}
4   metrics:
5     ipv4_address: 172.19.1.3
6 # Start GNB container after 5gc is up and running
7 depends_on:
8   5gc:
9     condition: service_healthy
10 # Command to run into the final container
11 command: gnb -c /gnb_config.yml -c /gnb_compose_config.yml
12 configs:
13   gnb_config.yml:
14     file: ${GNB_CONFIG_PATH:-../configs/
15           gnb_rf_b200_tdd_n78_20mhz.yml}
16   gnb_compose_config.yml:
17     content: |
18       cu_cp:
19         amf:
20           addrs: ${OPEN5GS_IP:-10.53.1.2}
21           addrs: ${OPEN5GS_IP:-192.168.152.131}
22           bind_addrs: ${GNB_IP:-10.53.1.3}
23           bind_addrs: ${GNB_IP:-192.168.152.140}
24       metrics:
25         autostart_stdout_metrics: true
26         enable_json: true
27         remote_control:
28           bind_addr: 0.0.0.0
29           enabled: true
30 volumes:
31   gnb-storage:
32 #networks:
33 #  ran:
34 #    ipam:
35 #      driver: default
36 #      config:
37 #        - subnet: 10.53.1.0/24
38 #  metrics:
39 #    ipam:
40 #      driver: default
41 #      config:
42 #        - subnet: 172.19.1.0/24
43 ..[codigo continua]..

```

6.8 Alterando arquivo .env (GRAFANA)

Para acessar o arquivo, basta seguir os caminhos abaixo para o host com o container da gnb:

```
1 cd ocudu_gnb/docker
```

```
2 nano .env
```

OBS: utilizando somente o comando `ls` para listar os arquivos na pasta docker não é possível visualizar o arquivo `.env`. Para isso basta utilizar o comando `ls -a`.

Como o container gnb funcionará em outro host, aqui deve ser colocado o IP do host que contém gnb. Dessa forma:

```
1 # Default: gNB running inside Docker (Option A)
2 WS_URL=gnb:8001
3 WS_URL=192.168.152.140:8001
4 ..[codigo continua]..
```